

Video Processing – HW3

Particle Filter Tracking – Part A

Course 0512.4263 2025/2026

Part A – Open Questions

Q1.

We saw that $\text{posterior} \propto \text{prior} \times \text{likelihood}$. Explain the formula and how it relates to the Kalman/Particle filter. Start from explaining what is the posterior, prior and likelihood.

Answer.

By Bayes' rule, $\text{posterior} = (\text{likelihood} \times \text{prior}) / \text{evidence}$. The evidence $p(z)$ does not depend on the state, so it is only a normalizing constant and we drop it, giving $\text{posterior} \propto \text{prior} \times \text{likelihood}$.

- **Prior** $p(x_t | z_{1:t-1})$: the belief about the state before seeing the new frame. We get it by pushing the previous estimate through the motion model (the predict step).
- **Likelihood** $p(z_t | x_t)$: how well a candidate state fits the new measurement. In our tracker this is the histogram similarity between the candidate's patch and the reference template q .
- **Posterior** $p(x_t | z_{1:t})$: the belief after using the new frame. It is large for states that the prior expected and the likelihood supports.

Both filters compute this. Kalman assumes every distribution is Gaussian and updates the mean and covariance analytically. The particle filter drops that assumption: the prior is the set of predicted particles, the likelihood becomes each particle's weight, and the posterior is the weighted (then resampled) particle set, so it can take any shape instead of only a Gaussian.

Q2.

For the measurement step in the particle filter, we used the histogram of the two patches (the patch around the new particle and the original patch of the object).

a. Why do we use the histogram to score the patches? what are the pros and cons of using this method?

Answer.

A histogram captures the colour distribution of the patch and deliberately ignores where each pixel sits.

- **Pros:** robust to small shifts, rotation, pose/shape change and partial occlusion (a person is non-rigid, so this matters); cheap to compute and compare; gives a compact, normalized appearance signature.

- **Cons:** throws away all spatial / structural information — two different objects with the same colours look identical; sensitive to illumination and colour changes; background pixels that fall inside the box pollute the histogram; quantizing to 4 bits loses fine colour detail.

b. We took the patches and computed the histogram and their distances. Why didn't we use the SSD (sum square difference) between the patches (in the image space)?

Answer.

SSD compares pixels position-by-position, so it assumes the two patches are perfectly aligned. A one-pixel shift, a small rotation, a scale change or any deformation makes SSD explode — even for the same object.

c. Can you think of another method to compare the patches? What are the pros and cons of the new method?

You can describe an example if it helps you.

Answer.

As we saw in Q2.a, one of the cons of the normal histogram is that it loses all spatial information. To improve that we can use a block color histogram: split the box into a grid (2x2 or 3x3 cells), compute one histogram per cell, compare cell-to-cell, and then sum the distances.

- **Pros:** more discriminative than the global histogram; still keeps most of the histogram robustness (small shifts, partial occlusion).
- **Cons:** less tolerant to deformation/rotation than the global histogram (cells must roughly align); more bins means a bit more compute.

Example: two people, one with a red shirt over blue jeans and the other blue over red, give the same global histogram but clearly different block histograms.

Q3.

Will particle filter work when the tracked object changes its scale (like a person walking towards and away from the camera)? What about the viewpoint of the object (like a person dancing)?

Answer.

Scale (walking toward/away). When the tracked object changes its scale, the tracking box has to change size too, otherwise it can't stay on the object. In a simple particle filter (like in the coding question) the box size stays fixed for the whole run, so when the object grows or shrinks the tracking fails. If the box size were dynamic, able to grow and shrink with the object, the object would be tracked successfully.

Viewpoint (dancing). Viewpoint change means the object looks different even though it stays in the same place (a dancing person moving and turning). The filter matches each candidate box against a stored template of the object, so when the appearance changes the match gets worse. A small change is fine, since the object still looks similar, but a big one like turning around no longer matches the template and the tracking fails. The solution is to update the template over time instead of keeping the first-frame one, which is what Q4 deals with.

Q4.

Can you think about a way to know when to update the template? Think of a scenario where we track a person. We keep a patch of the object's appearance in the first frame, and then compare each new patch to it. One of the problems is that, as time passes, the original patch no longer represents the object (light, viewpoint, ...). Can you think of a way to determine when to update the patch of the object? And how to determine the new patch (based on the previous frame/ based on the last two frames/...)?

Answer.

When: use the best particle's match score (its weight / Bhattacharyya value). High score: keep the template. Score drops but still locked on: appearance drifted (light, viewpoint), so update. Score very low: do *not* update, since that is usually occlusion and we would learn the background.

How: take the patch under the best (max-weight) particle and blend it in rather than replacing it, $q_{new} = (1 - \alpha) q_{old} + \alpha q_{current}$ with small α , so it adapts slowly and one bad frame cannot ruin it.